

MÓDULOS DE SOFTWARE PROBADOS

Proyecto:

BIDA – Odontología Especializada

Evidencia:

Módulos de software probados

Aprendiz:

Carlos Andrés Cajas Cerón

Ficha:

3118326

Tutor:

Gilber Ceron Muñoz

Programa de formación:

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje – SENA

Año:

2026

1. INTRODUCCIÓN

El presente documento tiene como finalidad presentar las pruebas realizadas a los módulos de software desarrollados para el proyecto BIDA – Odontología Especializada, con el propósito de verificar el correcto funcionamiento de las funcionalidades implementadas y validar el comportamiento del sistema ante diferentes situaciones.

Para la realización de las pruebas se utilizó un conjunto de pruebas automatizadas orientadas a comprobar las operaciones principales de los módulos, incluyendo consultas, creación, actualización y eliminación de registros, además del manejo de situaciones de error y validaciones de datos.

Las pruebas fueron desarrolladas utilizando Jest, junto con las herramientas utilizadas en el proyecto para realizar las pruebas de las rutas de la API desarrollada con Node.js y Express.

Como resultado de la ejecución general de las pruebas se obtuvieron 10 suites de pruebas aprobadas y 132 pruebas aprobadas de 132 pruebas ejecutadas, lo que permite evidenciar que los módulos evaluados cumplen con los comportamientos establecidos en las pruebas realizadas.

Este proceso permite identificar posibles errores durante el desarrollo y comprobar que las funcionalidades principales del sistema respondan de manera adecuada tanto en escenarios exitosos como ante situaciones de error.

2. OBJETIVOS

2.1 Objetivo general

Verificar mediante pruebas de software el correcto funcionamiento de los módulos desarrollados para el sistema BIDA – Odontología Especializada, comprobando que las funcionalidades implementadas respondan correctamente ante diferentes escenarios de uso y manejo de errores.

2.2 Objetivos específicos

- Comprobar el funcionamiento de las principales rutas de la API desarrollada para el sistema BIDA.
- Verificar las operaciones de consulta, creación, actualización y eliminación de información.
- Validar el comportamiento del sistema cuando se presentan datos incompletos o registros inexistentes.
- Comprobar el manejo de errores relacionados con la conexión o comunicación con la base de datos.
- Ejecutar pruebas automatizadas utilizando Jest para verificar el comportamiento de los módulos.

- Registrar los resultados obtenidos durante la ejecución de las pruebas.
- Identificar mediante las pruebas posibles errores y verificar su correspondiente manejo dentro de la aplicación.
- Evidenciar que los módulos probados presentan un comportamiento satisfactorio de acuerdo con los casos de prueba implementados.

3. DESCRIPCIÓN DEL PROYECTO

BIDA – Odontología Especializada es un sistema de software orientado a apoyar la gestión de los procesos relacionados con la atención odontológica.

El proyecto cuenta con una estructura de base de datos y una API desarrollada utilizando tecnologías como Node.js, Express y MySQL, permitiendo realizar operaciones sobre la información almacenada en la base de datos.

Dentro de la solución se encuentran diferentes módulos relacionados con la gestión de la información del sistema, entre ellos pacientes, odontólogos, empleados, citas, tratamientos, detalle de tratamientos, consultorios, locales y facturación.

Para verificar el funcionamiento de estos módulos se implementaron pruebas automatizadas sobre las rutas de la aplicación. Estas pruebas permiten comprobar tanto los escenarios de funcionamiento correcto como diferentes situaciones de error.

4. MÓDULOS DE SOFTWARE PRBADOS

Durante el proceso de validación se realizaron pruebas automatizadas sobre los módulos principales de la API del sistema BIDA – Odontología Especializada.

Los módulos evaluados fueron los siguientes:

1. Citas
2. Tratamientos
3. Facturación
4. Pacientes
5. Locales
6. Odontólogos
7. Empleados
8. Detalle de tratamientos
9. Consultorios

Adicionalmente, las pruebas fueron organizadas en diferentes archivos de prueba ("test") para verificar el comportamiento específico de cada conjunto de rutas.

Las pruebas incluyeron operaciones de consulta, creación, actualización y eliminación, así como validaciones de datos, registros inexistentes y manejo de errores de conexión con la base de datos.

5. TIPOS DE PRUEBAS REALIZADAS

Para validar el funcionamiento de los módulos del sistema BIDA se realizaron diferentes tipos de casos de prueba.

5.1 Pruebas de consulta

Se verificó que las rutas encargadas de consultar información permitan obtener correctamente los registros existentes.

También se verificó el comportamiento cuando no existen registros disponibles o cuando se solicita un registro que no se encuentra en la base de datos.

5.2 Pruebas de creación

Se realizaron pruebas para comprobar que los módulos permitan crear nuevos registros cuando los datos proporcionados son correctos.

También se probaron situaciones en las cuales se proporcionan datos incompletos o se intenta registrar información duplicada.

5.3 Pruebas de actualización

Se verificó que los registros existentes puedan ser actualizados correctamente y que el sistema valide situaciones como la ausencia de información necesaria o la actualización de un registro inexistente.

5.4 Pruebas de eliminación

Se verificó que los registros existentes puedan ser eliminados correctamente y que el sistema responda apropiadamente cuando se intenta eliminar un registro que no existe.

5.5 Pruebas de manejo de errores

Se realizaron pruebas provocando errores controlados, como errores de conexión con la base de datos, con el propósito de verificar que las rutas respondan con los códigos HTTP correspondientes y mensajes de error adecuados.

5.6 Pruebas de validación

Se verificó el comportamiento de las rutas cuando reciben información incompleta, incorrecta o que no cumple con las condiciones necesarias para realizar la operación solicitada.

6. HERRAMIENTA UTILIZADA PARA LAS PRUEBAS

Para la ejecución de las pruebas automatizadas se utilizó Jest, herramienta de pruebas para aplicaciones JavaScript.

Jest permitió ejecutar los diferentes archivos de pruebas creados para los módulos del proyecto y verificar automáticamente si cada caso cumplía con el resultado esperado.

La ejecución se realizó desde la terminal de Visual Studio Code utilizando el comando correspondiente para ejecutar las pruebas del proyecto.

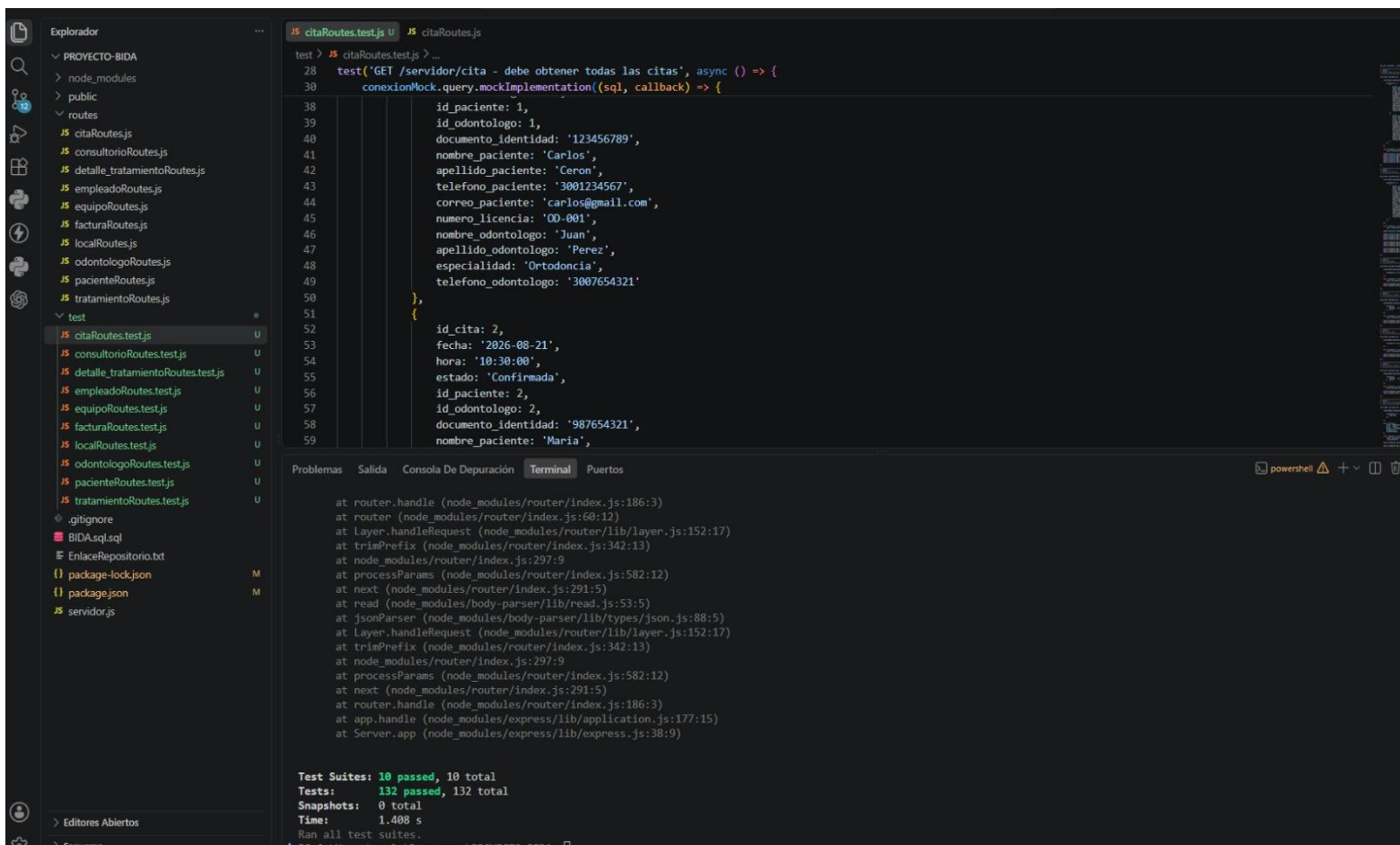
El resultado final de la ejecución general fue:

Test Suites: 10 passed, 10 total

Tests: 132 passed, 132 total

Snapshots: 0 total

Esto indica que las 10 suites de pruebas ejecutadas fueron aprobadas y que los 132 casos de prueba evaluados obtuvieron el resultado esperado.



7. PRUEBAS DEL MÓDULO DE CITAS

El módulo de citas permite gestionar la información relacionada con las citas odontológicas del sistema BIDA – Odontología Especializada.

Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las principales operaciones disponibles en las rutas del módulo.

Las pruebas permitieron comprobar las operaciones de consulta, creación, actualización y eliminación de citas, además de validar diferentes situaciones de error.

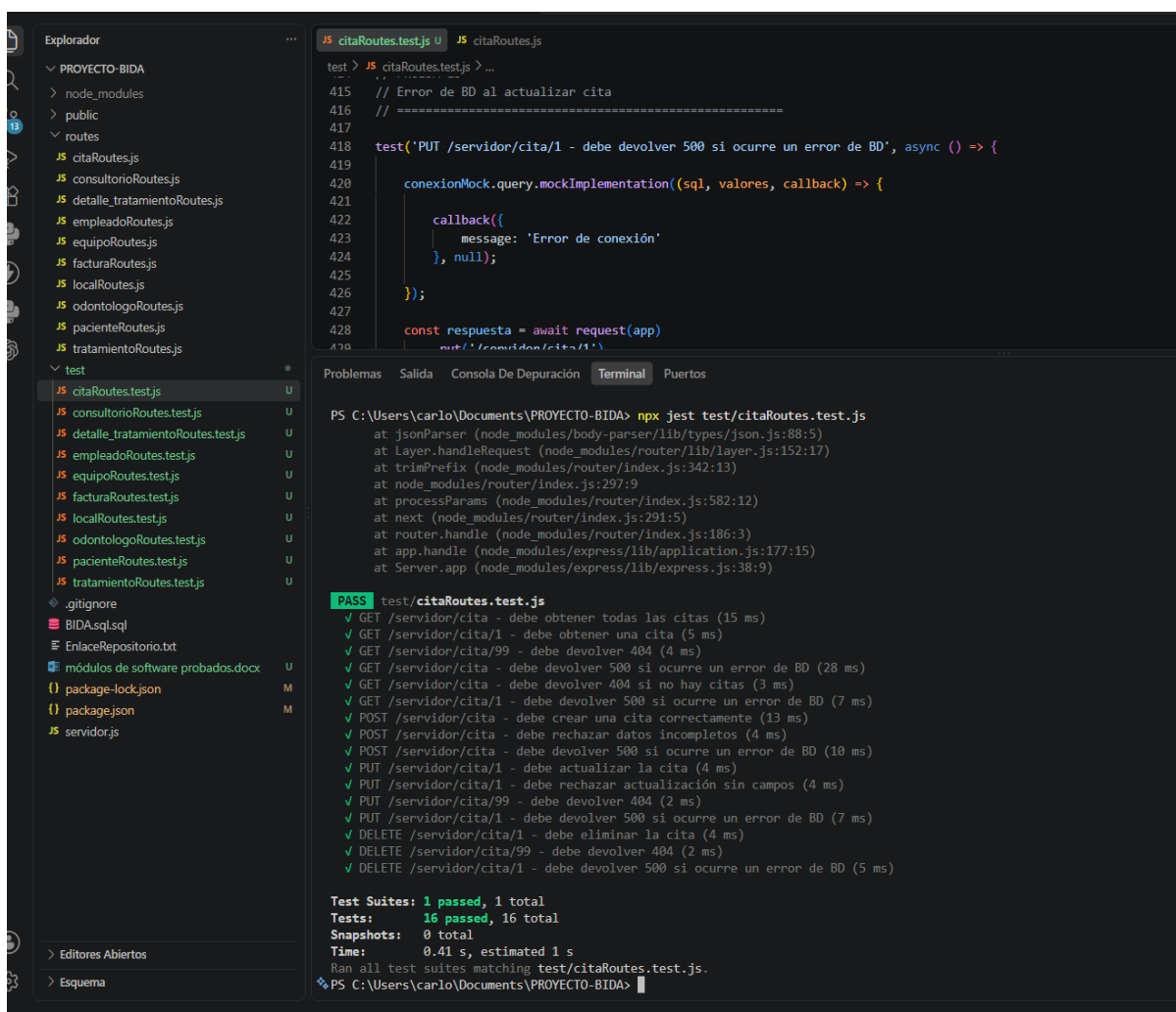
7.1 Casos de prueba realizados

Para el módulo de citas se realizaron los siguientes casos de prueba:

No.	Operación	Caso evaluado	Resultado
1	GET	Obtener todas las citas	Aprobado
2	GET	Obtener una cita existente	Aprobado
3	GET	Consultar una cita inexistente	Aprobado
4	GET	Error de conexión al consultar citas	Aprobado
5	GET	Consulta sin registros	Aprobado
6	GET	Error de conexión al consultar una cita	Aprobado
7	POST	Crear una cita correctamente	Aprobado

8	POST	Rechazar datos incompletos	Aprobado
9	POST	Error de conexión al crear una cita	Aprobado
10	PUT	Actualizar una cita	Aprobado
11	PUT	Rechazar actualización sin campos	Aprobado
12	PUT	Actualizar una cita inexistente	Aprobado
13	PUT	Error de conexión al actualizar una cita	Aprobado
14	DELETE	Eliminar una cita	Aprobado
15	DELETE	Eliminar una cita inexistente	Aprobado
16	DELETE	Error de conexión al eliminar una cita	Aprobado

Los casos de prueba permitieron verificar tanto los escenarios de funcionamiento normal como el manejo de situaciones excepcionales.



The screenshot shows the VS Code interface with the Explorer, Source Explorer, and Terminal panels. The Explorer panel on the left shows the project structure, including the 'test' directory. The Source Explorer panel in the middle shows the 'citaRoutes.test.js' file. The Terminal panel on the right displays the output of the Jest test run.

```
test > JS citaRoutes.test.js > ...
415 // Error de BD al actualizar cita
416 // =====
417
418 test('PUT /servidor/cita/1 - debe devolver 500 si ocurre un error de BD', async () => {
419
420     conexionMock.query.mockImplementation((sql, valores, callback) => {
421
422         callback({
423             message: 'Error de conexión'
424         }, null);
425     });
426
427     const respuesta = await request(app)
428         .put('/servidor/cita/1')
```

Problemas Salida Consola De Depuración Terminal Puertos

PS C:\Users\carlo\Documents\PROYECTO-BIDA> npx jest test/citaRoutes.test.js

at jsonParser (node_modules/body-parser/lib/types/json.js:88:5)
at Layer.handleRequest (node_modules/router/lib/layer.js:152:17)
at trimPrefix (node_modules/router/index.js:342:13)
at node_modules/router/index.js:297:9
at processParams (node_modules/router/index.js:582:12)
at next (node_modules/router/index.js:291:5)
at router.handle (node_modules/router/index.js:186:3)
at app.handle (node_modules/express/lib/application.js:177:15)
at Server.app (node_modules/express/lib/express.js:38:9)

PASS test/citaRoutes.test.js

- ✓ GET /servidor/cita - debe obtener todas las citas (15 ms)
- ✓ GET /servidor/cita/1 - debe obtener una cita (5 ms)
- ✓ GET /servidor/cita/99 - debe devolver 404 (4 ms)
- ✓ GET /servidor/cita - debe devolver 500 si ocurre un error de BD (28 ms)
- ✓ GET /servidor/cita - debe devolver 404 si no hay citas (3 ms)
- ✓ GET /servidor/cita/1 - debe devolver 500 si ocurre un error de BD (7 ms)
- ✓ POST /servidor/cita - debe crear una cita correctamente (13 ms)
- ✓ POST /servidor/cita - debe rechazar datos incompletos (4 ms)
- ✓ POST /servidor/cita - debe devolver 500 si ocurre un error de BD (10 ms)
- ✓ PUT /servidor/cita/1 - debe actualizar la cita (4 ms)
- ✓ PUT /servidor/cita/1 - debe rechazar actualización sin campos (4 ms)
- ✓ PUT /servidor/cita/99 - debe devolver 404 (2 ms)
- ✓ PUT /servidor/cita/1 - debe devolver 500 si ocurre un error de BD (7 ms)
- ✓ DELETE /servidor/cita/1 - debe eliminar la cita (4 ms)
- ✓ DELETE /servidor/cita/99 - debe devolver 404 (2 ms)
- ✓ DELETE /servidor/cita/1 - debe devolver 500 si ocurre un error de BD (5 ms)

Test Suites: 1 passed, 1 total
Tests: 16 passed, 16 total
Snapshots: 0 total
Time: 0.41 s, estimated 1 s
Ran all test suites matching test/citaRoutes.test.js.
PS C:\Users\carlo\Documents\PROYECTO-BIDA>

7.2 Resultado de las pruebas

Las pruebas correspondientes al módulo de citas fueron ejecutadas mediante Jest y finalizaron correctamente.

Durante las pruebas se verificaron las respuestas esperadas para las operaciones de consulta, creación, actualización y eliminación.

También se comprobaron escenarios en los que se presentan errores de conexión con la base de datos, registros inexistentes y datos incompletos.

El archivo de pruebas correspondiente al módulo fue ejecutado satisfactoriamente, obteniendo un resultado de PASS.

Esto permite evidenciar que las rutas evaluadas del módulo de citas responden de acuerdo con los comportamientos establecidos en los casos de prueba.

8. PRUEBAS DEL MÓDULO DE TRATAMIENTOS

El módulo de tratamientos permite gestionar la información correspondiente a los tratamientos odontológicos registrados en el sistema BIDA – Odontología Especializada.

Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las operaciones principales del módulo.

Las pruebas permitieron validar las consultas, creación, actualización y eliminación de tratamientos, además del comportamiento del sistema ante diferentes situaciones de error.

8.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de tratamientos contemplaron:

- Consulta de todos los tratamientos.
- Consulta de un tratamiento específico.
- Consulta de un tratamiento inexistente.
- Manejo de errores de conexión durante las consultas.
- Creación correcta de un tratamiento.
- Validación de datos requeridos para crear un tratamiento.
- Manejo de errores durante la creación.
- Actualización de tratamientos.
- Validación de actualizaciones sin información suficiente.
- Actualización de tratamientos inexistentes.
- Manejo de errores de conexión durante la actualización.
- Eliminación de tratamientos.
- Validación de eliminación de registros inexistentes.
- Manejo de errores durante la eliminación.

8.2 Resultado

Las pruebas automatizadas correspondientes al módulo de tratamientos fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Los casos de prueba relacionados con los escenarios correctos y los escenarios de error fueron aprobados, verificando el comportamiento esperado de las rutas del módulo.

The screenshot shows the Visual Studio Code editor with a project named 'PROYECTO-BIDA'. The file explorer on the left shows a directory structure with various route files and a 'test' directory. The 'test' directory contains several test files, including 'tratamientoRoutes.test.js', which is currently selected. The editor displays the code for 'tratamientoRoutes.test.js', which uses Jest for testing. The code includes imports for 'express', 'supertest', and 'tratamientoRoutes', a mock for the database connection, and a setup for an Express application. The test suite 'test/tratamientoRoutes.test.js' is shown as passed, with 13 individual tests succeeding. The terminal at the bottom shows the command 'npx jest test/tratamientoRoutes.test.js' being executed, and the output displays the test results, including the number of passed tests, total tests, and the time taken.

```
test > JS tratamientoRoutes.test.js > ...
1  const express = require('express');
2  const request = require('supertest');
3  const tratamientoRoutes = require('../routes/tratamientoRoutes');
4
5  // =====
6  // MOCK DE LA CONEXIÓN A MYSQL
7  // =====
8
9  const conexionMock = {
10   query: jest.fn()
11 };
12
13 // =====
14 // CONFIGURACIÓN DE EXPRESS
15 // =====
16
17 const app = express();
18
19 app.use(express.json());
20
```

Problemas Salida Consola De Depuración Terminal Puertos

PS C:\Users\carlo\Documents\PROYECTO-BIDA> npx jest test/tratamientoRoutes.test.js

at router.handle (node_modules/router/index.js:186:3)
at app.handle (node_modules/express/lib/application.js:177:15)
at Server.app (node_modules/express/lib/express.js:38:9)

PASS test/tratamientoRoutes.test.js

- ✓ GET /servidor/tratamiento - debe obtener todos los tratamientos (19 ms)
- ✓ GET /servidor/tratamiento/1 - debe obtener un tratamiento (5 ms)
- ✓ GET /servidor/tratamiento/99 - debe devolver 404 (4 ms)
- ✓ GET /servidor/tratamiento - debe devolver 500 si ocurre un error de BD (39 ms)
- ✓ POST /servidor/tratamiento - debe crear un tratamiento correctamente (18 ms)
- ✓ POST /servidor/tratamiento - debe rechazar datos incompletos (4 ms)
- ✓ POST /servidor/tratamiento - debe devolver 500 si ocurre un error de BD (11 ms)
- ✓ PUT /servidor/tratamiento/1 - debe actualizar el tratamiento (5 ms)
- ✓ PUT /servidor/tratamiento/1 - debe rechazar actualización sin campos (4 ms)
- ✓ PUT /servidor/tratamiento/99 - debe devolver 404 (5 ms)
- ✓ DELETE /servidor/tratamiento/2 - debe eliminar tratamiento (3 ms)
- ✓ DELETE /servidor/tratamiento/99 - debe devolver 404 (2 ms)
- ✓ DELETE /servidor/tratamiento/1 - debe devolver 500 si ocurre un error de BD (11 ms)

Test Suites: 1 passed, 1 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 0.548 s, estimated 1 s
Ran all test suites matching test/tratamientoRoutes.test.js.
PS C:\Users\carlo\Documents\PROYECTO-BIDA>

9. PRUEBAS DEL MÓDULO DE FACTURACIÓN

El módulo de facturación permite gestionar la información relacionada con las facturas generadas dentro del sistema BIDA – Odontología Especializada.

Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las principales operaciones disponibles en las rutas del módulo. Las pruebas permitieron comprobar el registro de facturas, la inserción de los detalles asociados a los tratamientos y la consulta de facturas, además del manejo de diferentes situaciones de error.

9.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de facturación contemplaron:

Registro correcto de una factura.

Manejo de errores de conexión durante el registro de una factura.

Inserción de los detalles asociados a una factura.

Manejo de errores durante la inserción de los detalles de la factura.

Consulta de una factura.

Manejo de errores de conexión durante la consulta de una factura.

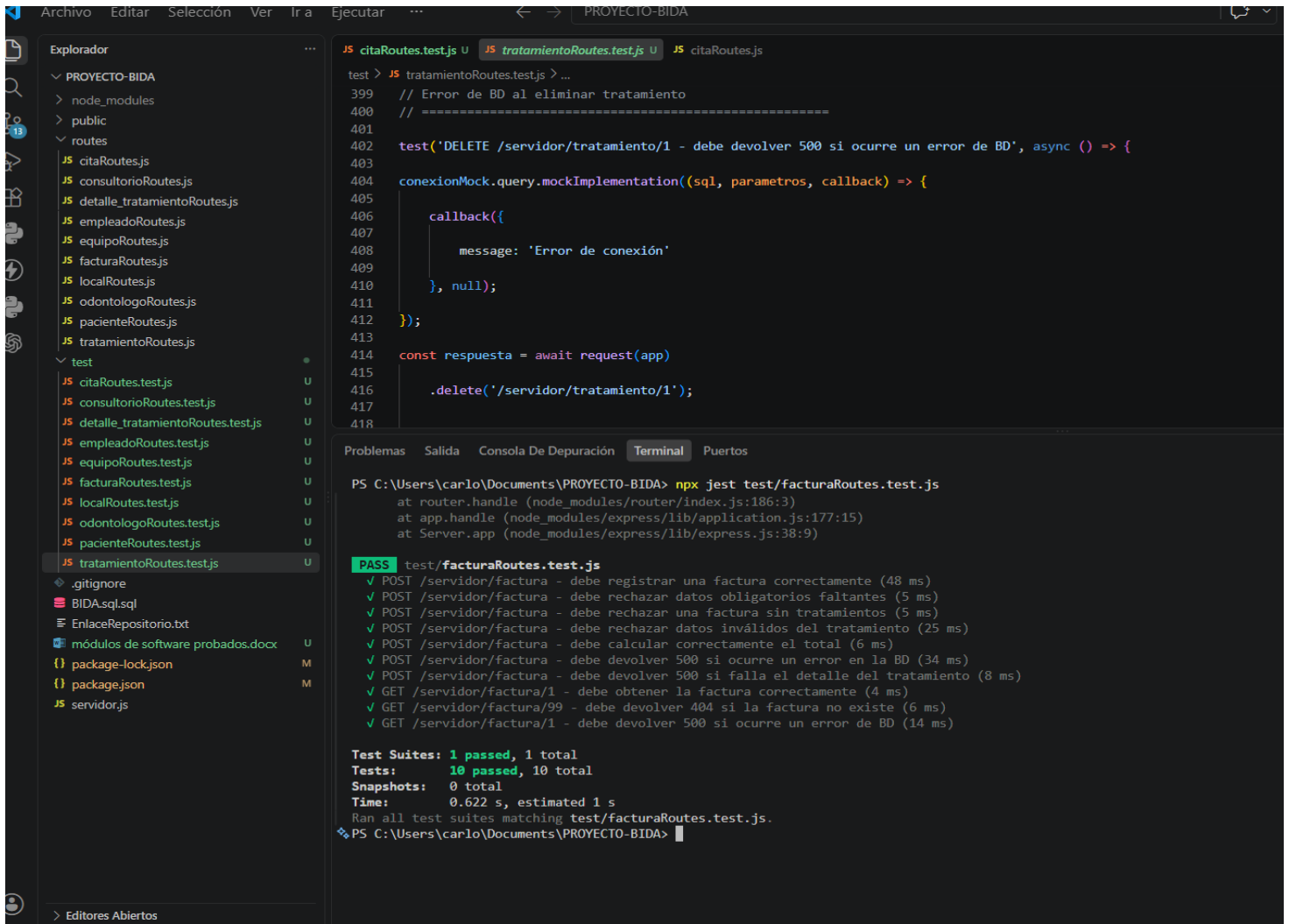
Los casos anteriores permitieron verificar el comportamiento esperado del módulo tanto en situaciones normales como en escenarios en los que se presentan errores durante las operaciones con la base de datos.

9.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de facturación fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Los casos de prueba verificaron el funcionamiento de las operaciones relacionadas con el registro, los detalles y la consulta de facturas.

También se comprobaron escenarios de error relacionados con la conexión a la base de datos y con la inserción de los detalles de una factura



The image shows a VS Code editor window with a project named "PROYECTO-BIDA". The left sidebar displays the file explorer with a tree structure: "PROYECTO-BIDA" > "node_modules" > "public" > "routes". The "routes" folder contains several JavaScript files, including "citaRoutes.js", "consultorioRoutes.js", "detalle_tratamientoRoutes.js", "empleadoRoutes.js", "equipoRoutes.js", "facturaRoutes.js", "localRoutes.js", "odontologoRoutes.js", "pacienteRoutes.js", and "tratamientoRoutes.js". The "test" folder contains corresponding test files, with "tratamientoRoutes.test.js" selected.

The main editor area shows the content of "tratamientoRoutes.test.js". It includes a Jest test suite for the DELETE endpoint "/servidor/tratamiento/1". The test suite uses a mock implementation for the database connection to simulate an error. The code is as follows:

```
test > JS tratamientoRoutes.test.js > ...
399 // Error de BD al eliminar tratamiento
400 // =====
401
402 test('DELETE /servidor/tratamiento/1 - debe devolver 500 si ocurre un error de BD', async () => {
403
404   conexionMock.query.mockImplementation((sql, parametros, callback) => {
405
406     callback({
407       message: 'Error de conexión'
408     }, null);
409   });
410
411   const respuesta = await request(app)
412     .delete('/servidor/tratamiento/1');
413
414   expect(respuesta.status).toBe(500);
415 });
```

The bottom panel shows the terminal output of the Jest test suite. The command executed is "npx jest test/facturaRoutes.test.js". The output shows that all tests passed, including tests for the DELETE endpoint. The test suite summary is as follows:

```
Test Suites: 1 passed, 1 total
Tests: 10 passed, 10 total
Snapshots: 0 total
Time: 0.622 s, estimated 1 s
Ran all test suites matching test/facturaRoutes.test.js.
```

The terminal also shows the command prompt "PS C:\Users\carlo\Documents\PROYECTO-BIDA>" and the command "npx jest test/facturaRoutes.test.js".

10. PRUEBAS DEL MÓDULO DE PACIENTES

El módulo de pacientes permite gestionar la información de los pacientes registrados en el sistema BIDA – Odontología Especializada. Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las operaciones principales disponibles en las rutas del módulo. Las pruebas permitieron comprobar el registro de pacientes y el manejo de diferentes situaciones relacionadas con la información almacenada en la base de datos.

10.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de pacientes contemplaron:

Creación correcta de un paciente.

Manejo de registros duplicados.

Validación de la respuesta del sistema cuando se presenta un error durante la inserción de información.

Verificación de la respuesta generada por el módulo ante diferentes situaciones de error.

Los casos permitieron comprobar que el módulo responde de acuerdo con los comportamientos establecidos para el registro de pacientes.

10.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de pacientes fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Durante las pruebas se verificó el comportamiento del sistema al registrar pacientes y al presentarse situaciones como registros duplicados o errores durante la comunicación con la base de datos.

```
test > JS tratamientoRoutes.test.js > ...
399 // Error de BD al eliminar tratamiento
400 // =====
401
402 test('DELETE /servidor/tratamiento/1 - debe devolver 500 si ocurre un error de BD', async () => {
403
404   conexionMock.query.mockImplementation((sql, parametros, callback) => {
405
406     callback({
407       message: 'Error de conexión'
408     }, null);
409   });
410
411   const respuesta = await request(app)
412     .delete('/servidor/tratamiento/1');
413
414   expect(respuesta.status).toBe(500);
415
416   // ...
417
418 }
```

```
PS C:\Users\carlo\Documents\PROYECTO-BIDA> npx jest test/pacienteRoutes.test.js
at router (node_modules/router/index.js:60:12)
at Layer.handleRequest (node_modules/router/lib/layer.js:152:17)
at trimPrefix (node_modules/router/index.js:342:13)
at node_modules/router/index.js:297:9
at processParams (node_modules/router/index.js:582:12)
at next (node_modules/router/index.js:291:5)
at node_modules/body-parser/lib/read.js:171:5
at invokeCallback (node_modules/raw-body/index.js:238:16)
at done (node_modules/raw-body/index.js:227:7)
at IncomingMessage.onEnd (node_modules/raw-body/index.js:287:7)

PASS test/pacienteRoutes.test.js
  ✓ GET /servidor/paciente - debe obtener todos los pacientes (23 ms)
  ✓ GET /servidor/paciente/1 - debe obtener al paciente (7 ms)
  ✓ GET /servidor/paciente/99 - debe indicar que no existe (4 ms)
  ✓ POST /servidor/crear-tabla-paciente - debe crear un paciente correctamente (44 ms)
  ✓ POST /servidor/crear-tabla-paciente - debe rechazar datos vacíos (5 ms)
  ✓ POST /servidor/crear-tabla-paciente - debe devolver 500 si hay error en BD (15 ms)
  ✓ PUT /servidor/paciente/1 - debe actualizar al paciente (5 ms)
  ✓ PUT /servidor/paciente/99 - debe devolver 404 (6 ms)
  ✓ DELETE /servidor/paciente/2 - debe eliminar paciente (4 ms)
  ✓ DELETE /servidor/paciente/99 - debe devolver 404 (3 ms)

Test Suites: 1 passed, 1 total
Tests: 10 passed, 10 total
Snapshots: 0 total
Time: 0.55 s, estimated 1 s
Ran all test suites matching test/pacienteRoutes.test.js.
PS C:\Users\carlo\Documents\PROYECTO-BIDA>
```

PRUEBAS DEL MÓDULO DE LOCALES

El módulo de locales permite gestionar la información correspondiente a los establecimientos o locales asociados al sistema BIDA – Odontología Especializada.

Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las operaciones principales de consulta, creación y eliminación de registros.

11.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de locales contemplaron:

Consulta de los locales registrados.

Manejo de errores de conexión durante la consulta.

Consulta de un local específico.

Manejo de errores durante la consulta de un local.

Creación de un local.

Manejo de errores de conexión durante la creación.

Eliminación de un local.

Manejo de errores de conexión durante la eliminación.

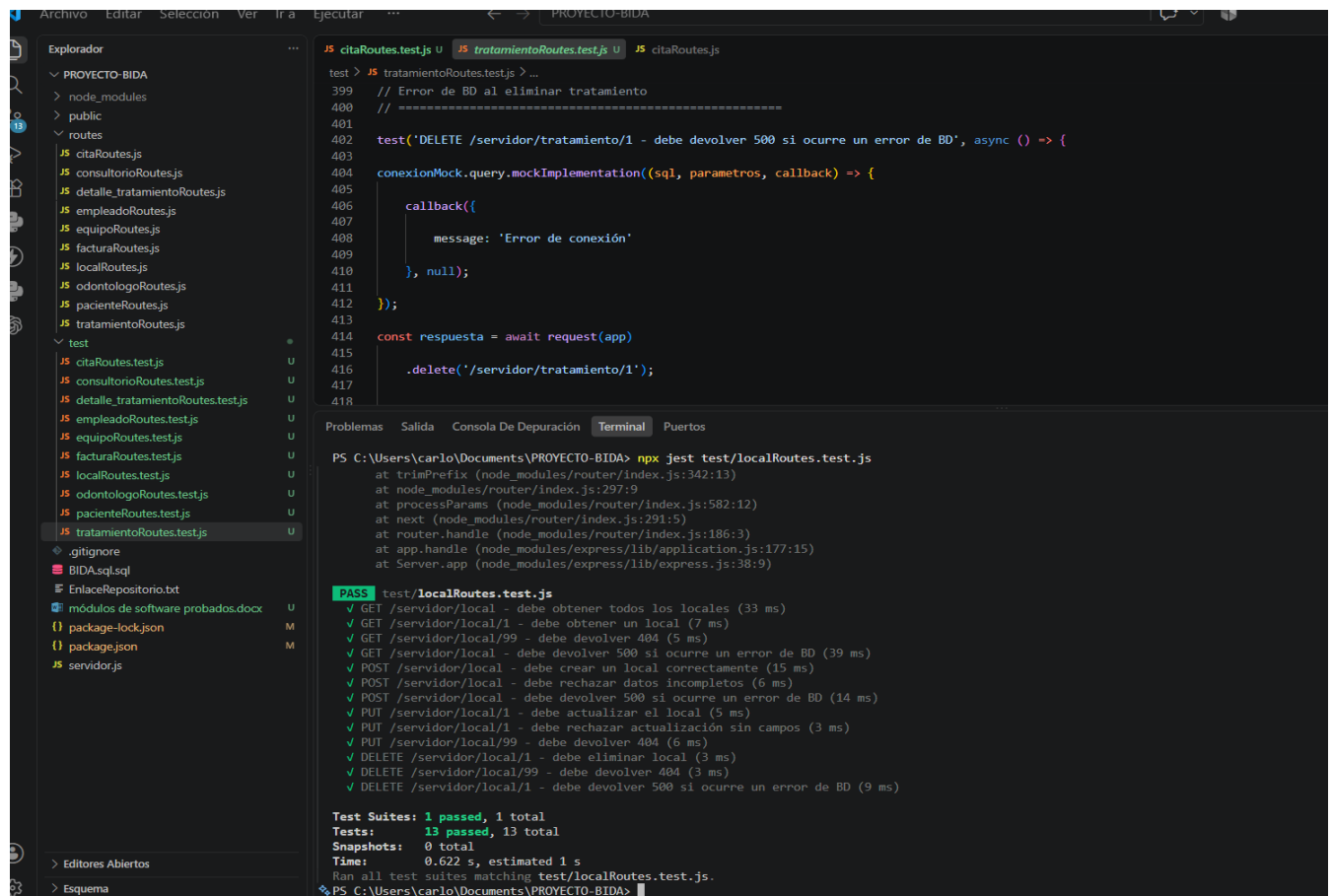
Estas pruebas permitieron comprobar el comportamiento de las rutas del módulo ante operaciones correctas y situaciones de error.

11.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de locales fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Se verificaron las respuestas generadas por el sistema durante las operaciones de consulta, creación y eliminación de locales.

También se evaluó el comportamiento de las rutas cuando se presentan errores durante la conexión con la base de datos.



```
PS C:\Users\carlo\Documents\PROYECTO-BIDA> npx jest test/localRoutes.test.js
at trimPrefix (node_modules/router/index.js:342:13)
at node_modules/router/index.js:297:9
at processParams (node_modules/router/index.js:582:12)
at next (node_modules/router/index.js:291:5)
at router.handle (node_modules/router/index.js:186:3)
at app.handle (node_modules/express/lib/application.js:177:15)
at Server.app (node_modules/express/lib/express.js:38:9)

PASS test/localRoutes.test.js
  ✓ GET /servidor/local - debe obtener todos los locales (33 ms)
  ✓ GET /servidor/local/1 - debe obtener un local (7 ms)
  ✓ GET /servidor/local/99 - debe devolver 404 (5 ms)
  ✓ GET /servidor/local - debe devolver 500 si ocurre un error de BD (39 ms)
  ✓ POST /servidor/local - debe crear un local correctamente (15 ms)
  ✓ POST /servidor/local - debe rechazar datos incompletos (6 ms)
  ✓ POST /servidor/local - debe devolver 500 si ocurre un error de BD (14 ms)
  ✓ PUT /servidor/local/1 - debe actualizar el local (5 ms)
  ✓ PUT /servidor/local/1 - debe rechazar actualización sin campos (3 ms)
  ✓ PUT /servidor/local/99 - debe devolver 404 (6 ms)
  ✓ DELETE /servidor/local/1 - debe eliminar local (3 ms)
  ✓ DELETE /servidor/local/99 - debe devolver 404 (3 ms)
  ✓ DELETE /servidor/local/1 - debe devolver 500 si ocurre un error de BD (9 ms)

Test Suites: 1 passed, 1 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 0.622 s, estimated 1 s
Ran all test suites matching test/localRoutes.test.js.
PS C:\Users\carlo\Documents\PROYECTO-BIDA>
```

12. PRUEBAS DEL MÓDULO DE ODONTÓLOGOS

El módulo de odontólogos permite gestionar la información de los profesionales encargados de prestar los servicios odontológicos dentro del sistema BIDA – Odontología Especializada.

Para comprobar su funcionamiento se realizaron pruebas automatizadas sobre las principales operaciones disponibles en las rutas del módulo.

12.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de odontólogos contemplaron:

Consulta de odontólogos.

Manejo de errores de conexión durante la consulta.

Creación de un odontólogo.

Manejo de registros duplicados.

Eliminación de un odontólogo.

Manejo de errores de conexión durante la eliminación.

Las pruebas permitieron comprobar que el módulo maneja correctamente tanto las operaciones exitosas como diferentes situaciones de error.

12.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de odontólogos fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Durante la ejecución se verificaron las respuestas esperadas para las operaciones de consulta, creación y eliminación de odontólogos.

También se comprobó el manejo de registros duplicados y errores de conexión con la base de datos.

The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'PROYECTO-BIDA' with a 'test' directory containing several test files. The code editor shows the content of 'tratamientoRoutes.test.js', which includes a test for a DELETE request that returns 500 on a database error. Below the code editor, the 'Terminal' tab shows the output of running 'npm test', which lists 13 passed tests and their durations. The tests include GET, POST, PUT, and DELETE requests for various endpoints, all of which passed successfully.

```
test > JS tratamientoRoutes.test.js > ...
399 // Error de BD al eliminar tratamiento
400 // =====
401
402 test('DELETE /servidor/tratamiento/1 - debe devolver 500 si ocurre un error de BD', async () => {
403
404   conexionMock.query.mockImplementation((sql, parametros, callback) => {
405
406     callback({
407       message: 'Error de conexión'
408     }, null);
409   });
410
411   const respuesta = await request(app)
412     .delete('/servidor/tratamiento/1');
413
414   expect(respuesta.status).toBe(500);
415
416   return respuesta;
417
418 });
```

Problemas Salida Consola De Depuración **Terminal** Puertos

PS C:\Users\carlo\Documents\PROYECTO-BIDA> npm test test/odontologoRoutes.test.js

at trimPrefix (node_modules/router/index.js:342:13)
at node_modules/router/index.js:297:9
at processParams (node_modules/router/index.js:582:12)
at next (node_modules/router/index.js:291:5)
at router.handle (node_modules/router/index.js:186:3)
at app.handle (node_modules/express/lib/application.js:177:15)
at Server.app (node_modules/express/lib/express.js:38:9)

PASS test/odontologoRoutes.test.js

- ✓ GET /servidor/odontologo - debe obtener todos los odontólogos (26 ms)
- ✓ GET /servidor/odontologo/1 - debe obtener al odontólogo (8 ms)
- ✓ GET /servidor/odontologo/99 - debe devolver 404 (4 ms)
- ✓ GET /servidor/odontologo - debe devolver 500 si ocurre un error de BD (42 ms)
- ✓ POST /servidor/odontologo - debe crear un odontólogo correctamente (16 ms)
- ✓ POST /servidor/odontologo - debe rechazar datos incompletos (5 ms)
- ✓ POST /servidor/odontologo - debe devolver 409 si ya existe (13 ms)
- ✓ PUT /servidor/odontologo/1 - debe actualizar al odontólogo (4 ms)
- ✓ PUT /servidor/odontologo/1 - debe rechazar actualización sin campos (4 ms)
- ✓ PUT /servidor/odontologo/99 - debe devolver 404 (4 ms)
- ✓ DELETE /servidor/odontologo/2 - debe eliminar odontólogo (3 ms)
- ✓ DELETE /servidor/odontologo/99 - debe devolver 404 (3 ms)
- ✓ DELETE /servidor/odontologo/1 - debe devolver 500 si ocurre un error (13 ms)

Test Suites: 1 passed, 1 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 0.63 s, estimated 1 s
Ran all test suites matching test/odontologoRoutes.test.js.

PS C:\Users\carlo\Documents\PROYECTO-BIDA>

13. PRUEBAS DEL MÓDULO DE EMPLEADOS

El módulo de empleados permite gestionar la información correspondiente a los empleados registrados en el sistema BIDA – Odontología Especializada.

Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las operaciones implementadas en las rutas del módulo.

13.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de empleados contemplaron:

Registro de empleados.

Manejo de registros duplicados.

Validación de las respuestas generadas ante errores durante la creación.

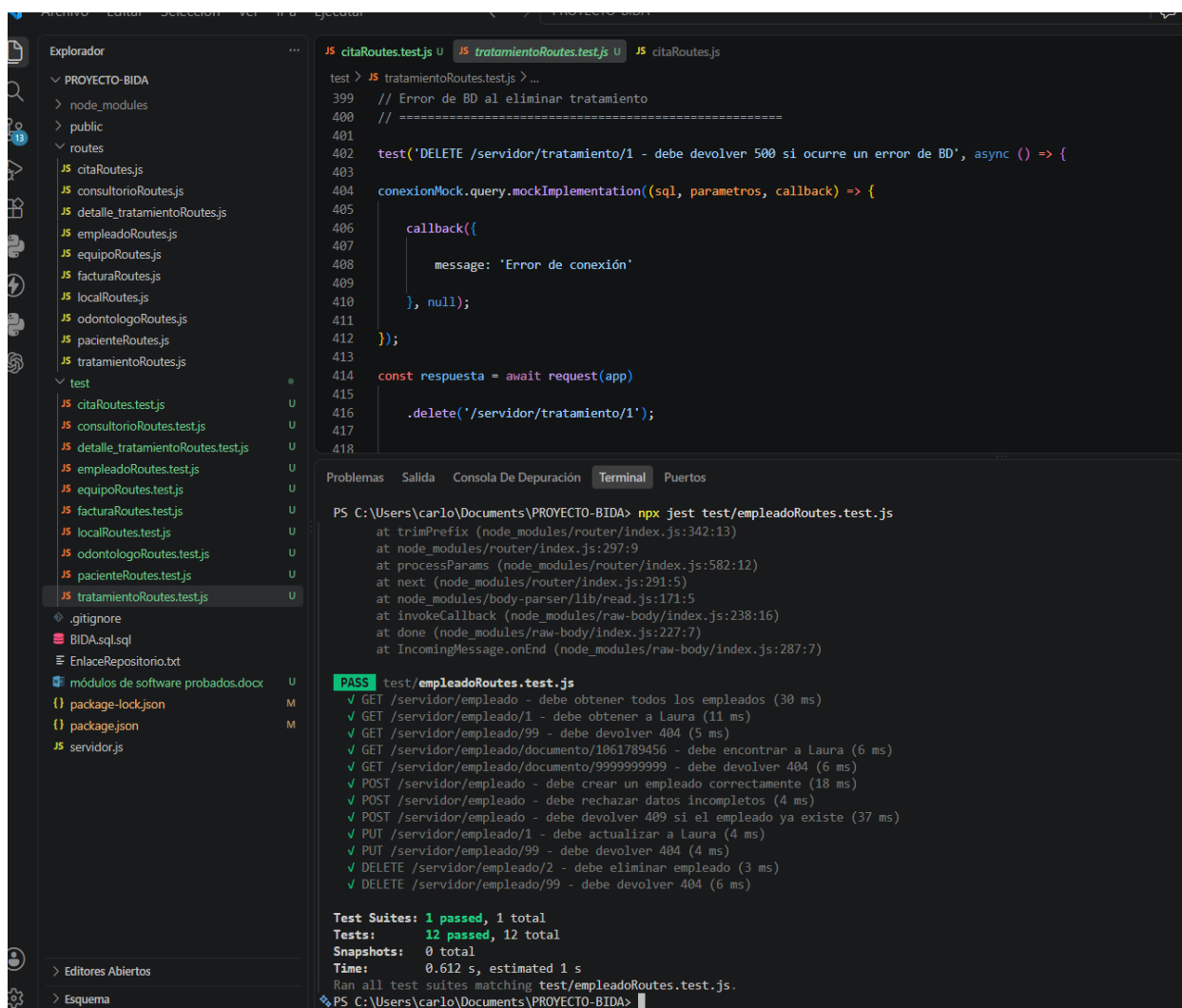
Verificación del comportamiento de las rutas del módulo ante diferentes escenarios.

Estas pruebas permitieron comprobar que el sistema responde adecuadamente durante el proceso de gestión de empleados.

13.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de empleados fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Se verificó principalmente el comportamiento de la ruta de creación de empleados y el manejo de registros duplicados.



The screenshot shows the Visual Studio Code interface with a project named 'PROYECTO-BIDA'. The file explorer on the left shows the project structure, including a 'test' directory with various route test files. The main editor displays the code for 'tratamientoRoutes.test.js', which includes a Jest test for a DELETE endpoint that expects a 500 status code in case of a database error. Below the code, the 'Terminal' tab shows the output of running 'npm test' for 'empleadoroutes.test.js'. The output lists 12 individual test cases, all of which passed. At the bottom, a summary shows 'Test Suites: 1 passed, 1 total' and 'Tests: 12 passed, 12 total'. The command prompt at the bottom shows the current directory as 'C:\Users\carlo\Documents\PROYECTO-BIDA'.

```
test > JS tratamientoRoutes.test.js > ...
399 // Error de BD al eliminar tratamiento
400 // =====
401
402 test('DELETE /servidor/tratamiento/1 - debe devolver 500 si ocurre un error de BD', async () => {
403
404   conexionMock.query.mockImplementation((sql, parametros, callback) => {
405
406     callback({
407
408       message: 'Error de conexión'
409
410     }, null);
411
412   });
413
414   const respuesta = await request(app)
415
416     .delete('/servidor/tratamiento/1');
417
418
```

Problemas Salida Consola De Depuración Terminal Puertos

PS C:\Users\carlo\Documents\PROYECTO-BIDA> npm test test/empleadoroutes.test.js

at trimPrefix (node_modules/router/index.js:342:13)
at node_modules/router/index.js:297:9
at processParams (node_modules/router/index.js:582:12)
at next (node_modules/router/index.js:291:5)
at node_modules/body-parser/lib/read.js:171:5
at invokeCallback (node_modules/raw-body/index.js:238:16)
at done (node_modules/raw-body/index.js:227:7)
at IncomingMessage.onEnd (node_modules/raw-body/index.js:287:7)

PASS test/empleadoroutes.test.js

- ✓ GET /servidor/empleado - debe obtener todos los empleados (30 ms)
- ✓ GET /servidor/empleado/1 - debe obtener a Laura (11 ms)
- ✓ GET /servidor/empleado/99 - debe devolver 404 (5 ms)
- ✓ GET /servidor/empleado/documento/1061789456 - debe encontrar a Laura (6 ms)
- ✓ GET /servidor/empleado/documento/9999999999 - debe devolver 404 (6 ms)
- ✓ POST /servidor/empleado - debe crear un empleado correctamente (18 ms)
- ✓ POST /servidor/empleado - debe rechazar datos incompletos (4 ms)
- ✓ POST /servidor/empleado - debe devolver 409 si el empleado ya existe (37 ms)
- ✓ PUT /servidor/empleado/1 - debe actualizar a Laura (4 ms)
- ✓ PUT /servidor/empleado/99 - debe devolver 404 (4 ms)
- ✓ DELETE /servidor/empleado/2 - debe eliminar empleado (3 ms)
- ✓ DELETE /servidor/empleado/99 - debe devolver 404 (6 ms)

Test Suites: 1 passed, 1 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Time: 0.612 s, estimated 1 s
Ran all test suites matching test/empleadoroutes.test.js.
PS C:\Users\carlo\Documents\PROYECTO-BIDA>

14. PRUEBAS DEL MÓDULO DE DETALLE DE TRATAMIENTOS

El módulo de detalle de tratamientos permite gestionar la información relacionada con los tratamientos asociados dentro del sistema BIDA – Odontología Especializada.

Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las operaciones principales de consulta, creación y eliminación de detalles.

14.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de detalle de tratamientos contemplaron:

- Consulta de los detalles registrados.

- Manejo de errores de conexión durante la consulta.

- Consulta de un detalle específico.

- Manejo de errores durante la consulta de un detalle.

- Creación de un detalle de tratamiento.

- Manejo de errores durante la creación.

- Eliminación de un detalle de tratamiento.

- Manejo de errores durante la eliminación.

Las pruebas permitieron comprobar el comportamiento de las rutas ante operaciones correctas y diferentes situaciones de error.

14.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de detalle de tratamientos fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Se verificaron las operaciones relacionadas con la consulta, creación y eliminación de los detalles de tratamientos.

Además, se comprobó el manejo de errores de conexión con la base de datos.

The image shows a VS Code editor interface. On the left, the Explorer sidebar displays the project structure for 'PROYECTO-BIDA', including folders like 'node_modules', 'public', and 'routes'. The 'routes' folder contains several test files, with 'tratamientoRoutes.test.js' selected. The main editor area shows the code for 'tratamientoRoutes.test.js', which includes a 'test' function for a DELETE request and a 'const respuesta = await request(app).delete...' line. The bottom panel shows the 'Terminal' tab with the command 'PS C:\Users\carlo\Documents\PROYECTO-BIDA> npx jest test/detalle_tratamientoRoutes.test.js' and its output, which lists 15 tests passed and the total execution time.

15. PRUEBAS DEL MÓDULO DE CONSULTORIOS

El módulo de consultorios permite gestionar la información correspondiente a los consultorios disponibles en el sistema BIDA – Odontología Especializada.

Para comprobar su funcionamiento se realizaron pruebas automatizadas sobre las principales operaciones disponibles en las rutas del módulo.

15.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de consultorios contemplaron:

- Consulta de los consultorios registrados.

- Manejo de errores de conexión durante la consulta.

- Consulta de un consultorio específico.

- Manejo de errores durante la consulta.

- Creación de un consultorio.

- Manejo de errores durante la creación.

- Eliminación de un consultorio.

- Manejo de errores durante la eliminación.

Estas pruebas permitieron verificar el comportamiento de las rutas del módulo ante operaciones normales y situaciones de error.

15.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de consultorios fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Durante las pruebas se verificaron las respuestas generadas por el sistema para las operaciones de consulta, creación y eliminación de consultorios.

También se evaluó el manejo de errores relacionados con la conexión a la base de datos

Archivo Editar Selección Ver Ir a Ejecutar ... PROYECTO-BIDA

Explorador

- PROYECTO-BIDA
 - node_modules
 - public
 - routes
 - citaRoutes.js
 - consultorioRoutes.js
 - detalle_tratamientoRoutes.js
 - empleadoRoutes.js
 - equipoRoutes.js
 - facturaRoutes.js
 - localRoutes.js
 - odontologoRoutes.js
 - pacienteRoutes.js
 - tratamientoRoutes.js
 - test
 - citaRoutes.test.js
 - consultorioRoutes.test.js
 - detalle_tratamientoRoutes.test.js
 - empleadoRoutes.test.js
 - equipoRoutes.test.js
 - facturaRoutes.test.js
 - localRoutes.test.js
 - odontologoRoutes.test.js
 - pacienteRoutes.test.js
 - tratamientoRoutes.test.js
 - .gitignore
 - BIDA.sqlsqli
 - EnlaceRepositorio.txt
 - módulos de software probados.docx
 - package-lock.json
 - package.json
 - servidor.js

JS citaRoutes.test.js JS tratamientoRoutes.test.js JS citaRoutes.js

```
test > JS tratamientoRoutes.test.js > ...
399 // Error de BD al eliminar tratamiento
400 // =====
401
402 test('DELETE /servidor/tratamiento/1 - debe devolver 500 si ocurre un error de BD', async () => {
403
404     conexionMock.query.mockImplementation((sql, parametros, callback) => {
405
406         callback({
407             message: 'Error de conexión'
408         }, null);
409     });
410
411     const respuesta = await request(app)
412         .delete('/servidor/tratamiento/1');
413
414     expect(respuesta.status).toBe(500);
415
416     // =====
417
418     // =====
419 }
```

Problemas Salida Consola De Depuración Terminal Puertos

PS C:\Users\carlo\Documents\PROYECTO-BIDA> npx jest test/consultorioRoutes.test.js

at next (node_modules/router/index.js:291:5)
at router.handle (node_modules/router/index.js:186:3)
at app.handle (node_modules/express/lib/application.js:177:15)
at Server.app (node_modules/express/lib/express.js:38:9)

PASS test/consultorioRoutes.test.js

- ✓ GET /servidor/consultorio - debe obtener todos los consultorios (19 ms)
- ✓ GET /servidor/consultorio/1 - debe obtener un consultorio (6 ms)
- ✓ GET /servidor/consultorio/99 - debe devolver 404 (5 ms)
- ✓ GET /servidor/consultorio - debe devolver 500 si ocurre un error de BD (41 ms)
- ✓ GET /servidor/consultorio - debe devolver 404 si no hay consultorios (6 ms)
- ✓ GET /servidor/consultorio/1 - debe devolver 500 si ocurre un error de BD (15 ms)
- ✓ POST /servidor/consultorio - debe crear un consultorio correctamente (19 ms)
- ✓ POST /servidor/consultorio - debe rechazar datos incompletos (4 ms)
- ✓ POST /servidor/consultorio - debe rechazar consultorio sin id_numero (4 ms)
- ✓ POST /servidor/consultorio - debe devolver 500 si ocurre un error de BD (11 ms)
- ✓ PUT /servidor/consultorio/1 - debe actualizar el consultorio (3 ms)
- ✓ PUT /servidor/consultorio/1 - debe rechazar actualización sin campos (3 ms)
- ✓ PUT /servidor/consultorio/99 - debe devolver 404 (4 ms)
- ✓ DELETE /servidor/consultorio/1 - debe eliminar el consultorio (3 ms)
- ✓ DELETE /servidor/consultorio/99 - debe devolver 404 (3 ms)
- ✓ DELETE /servidor/consultorio/1 - debe devolver 500 si ocurre un error de BD (13 ms)

Test Suites: 1 passed, 1 total
Tests: 16 passed, 16 total
Snapshots: 0 total
Time: 0.575 s, estimated 1 s
Ran all test suites matching test/consultorioRoutes.test.js.

PS C:\Users\carlo\Documents\PROYECTO-BIDA>

16. PRUEBAS DEL MÓDULO DE EQUIPOS

El módulo de equipos permite gestionar la información correspondiente a los equipos utilizados o registrados dentro del sistema BIDA – Odontología Especializada.

Para verificar su funcionamiento se realizaron pruebas automatizadas sobre las operaciones implementadas en las rutas correspondientes al módulo.

16.1 Casos de prueba

Los casos de prueba realizados sobre el módulo de equipos contemplaron:

- Consulta de los equipos registrados.

- Consulta de un equipo específico.

- Manejo de situaciones en las que no se encuentra un equipo.

- Creación de un equipo.

- Validación de los datos utilizados para registrar un equipo.

- Actualización de un equipo.

- Validación de la actualización de información.

- Eliminación de un equipo.

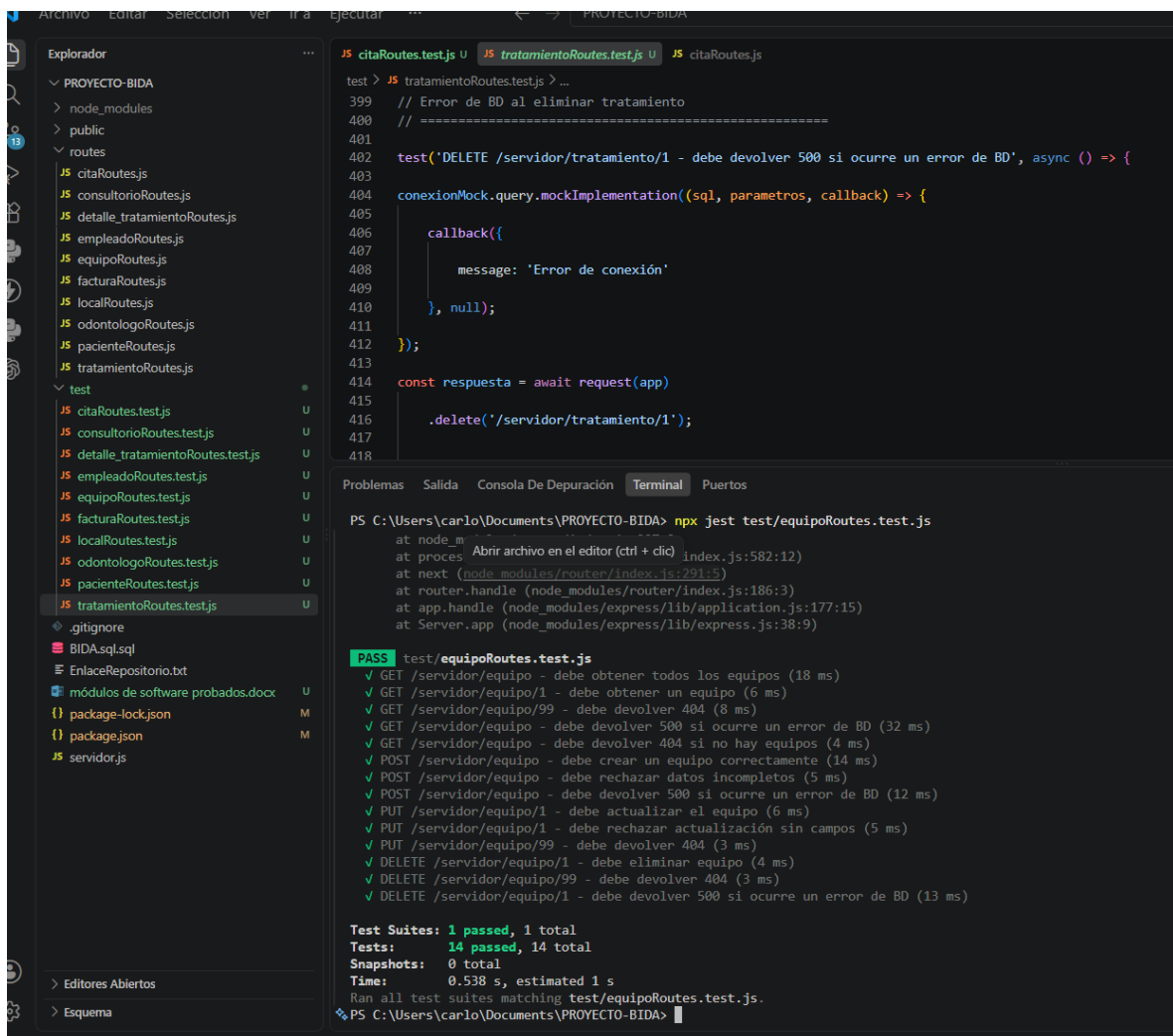
- Manejo de diferentes situaciones de error durante las operaciones.

Estas pruebas permitieron verificar el comportamiento de las rutas del módulo y comprobar que las respuestas generadas por el sistema corresponden a los escenarios evaluados.

16.2 Resultado de las pruebas

Las pruebas automatizadas correspondientes al módulo de equipos fueron ejecutadas mediante Jest y finalizaron satisfactoriamente.

Durante las pruebas se verificaron las operaciones disponibles para la gestión de los equipos, incluyendo las situaciones normales y los escenarios de error contemplados en el archivo de pruebas.



The screenshot shows a VS Code editor with a project named 'PROYECTO-BIDA'. The Explorer panel on the left shows the file structure, including a 'routes' directory with various route files and a 'test' directory with corresponding test files. The 'trataimientoRoutes.test.js' file is selected. The main editor shows the code for this test file, which includes a Jest test for a DELETE request to '/servidor/tratamiento/1'. The test uses a mock implementation for the database query to simulate an error. The bottom panel shows the 'Terminal' output, which displays the command 'npx jest test/equipoRoutes.test.js' and the results of the test suite. The test suite 'test/equipoRoutes.test.js' passed, with 14 individual tests passing. The total time for the test suite was 0.538 seconds.

```
test > JS trataimientoRoutes.test.js > ...
399 // Error de BD al eliminar tratamiento
400 //
401
402 test('DELETE /servidor/tratamiento/1 - debe devolver 500 si ocurre un error de BD', async () => {
403
404   conexionMock.query.mockImplementation((sql, parametros, callback) => {
405
406     callback({
407       message: 'Error de conexión'
408     }, null);
409   });
410
411   const respuesta = await request(app)
412     .delete('/servidor/tratamiento/1');
413
414   expect(respuesta.status).toBe(500);
415
416   // ...
417
418 }
```

Problemas Salida Consola De Depuración Terminal Puertos

PS C:\Users\carlo\Documents\PROYECTO-BIDA> npx jest test/equipoRoutes.test.js

at node_modules\jest\build\testScheduler\testScheduler.js:100:13
at processTicksAndRejections (node_modules\process\src\process.js:29:11)
at next (node_modules\router\index.js:291:5)
at router.handle (node_modules\router\index.js:186:3)
at app.handle (node_modules\express\lib\application.js:177:15)
at Server.app (node_modules\express\lib\express.js:38:9)

PASS test/equipoRoutes.test.js

- ✓ GET /servidor/equipo - debe obtener todos los equipos (18 ms)
- ✓ GET /servidor/equipo/1 - debe obtener un equipo (6 ms)
- ✓ GET /servidor/equipo/99 - debe devolver 404 (8 ms)
- ✓ GET /servidor/equipo - debe devolver 500 si ocurre un error de BD (32 ms)
- ✓ GET /servidor/equipo - debe devolver 404 si no hay equipos (4 ms)
- ✓ POST /servidor/equipo - debe crear un equipo correctamente (14 ms)
- ✓ POST /servidor/equipo - debe rechazar datos incompletos (5 ms)
- ✓ POST /servidor/equipo - debe devolver 500 si ocurre un error de BD (12 ms)
- ✓ PUT /servidor/equipo/1 - debe actualizar el equipo (6 ms)
- ✓ PUT /servidor/equipo/1 - debe rechazar actualización sin campos (5 ms)
- ✓ PUT /servidor/equipo/99 - debe devolver 404 (3 ms)
- ✓ DELETE /servidor/equipo/1 - debe eliminar equipo (4 ms)
- ✓ DELETE /servidor/equipo/99 - debe devolver 404 (3 ms)
- ✓ DELETE /servidor/equipo/1 - debe devolver 500 si ocurre un error de BD (13 ms)

Test Suites: 1 passed, 1 total
Tests: 14 passed, 14 total
Snapshots: 0 total
Time: 0.538 s, estimated 1 s
Ran all test suites matching test/equipoRoutes.test.js.

PS C:\Users\carlo\Documents\PROYECTO-BIDA>

17. RESULTADOS GENERALES DE LAS PRUEBAS

Una vez ejecutadas las pruebas automatizadas correspondientes a los módulos desarrollados en el sistema BIDA – Odontología

Especializada, se realizó una ejecución general de todos los archivos de prueba mediante Jest.

La ejecución permitió verificar de manera conjunta el funcionamiento de los módulos y las diferentes rutas implementadas en el proyecto.

16.1 Resultado general

El resultado obtenido durante la ejecución general fue:

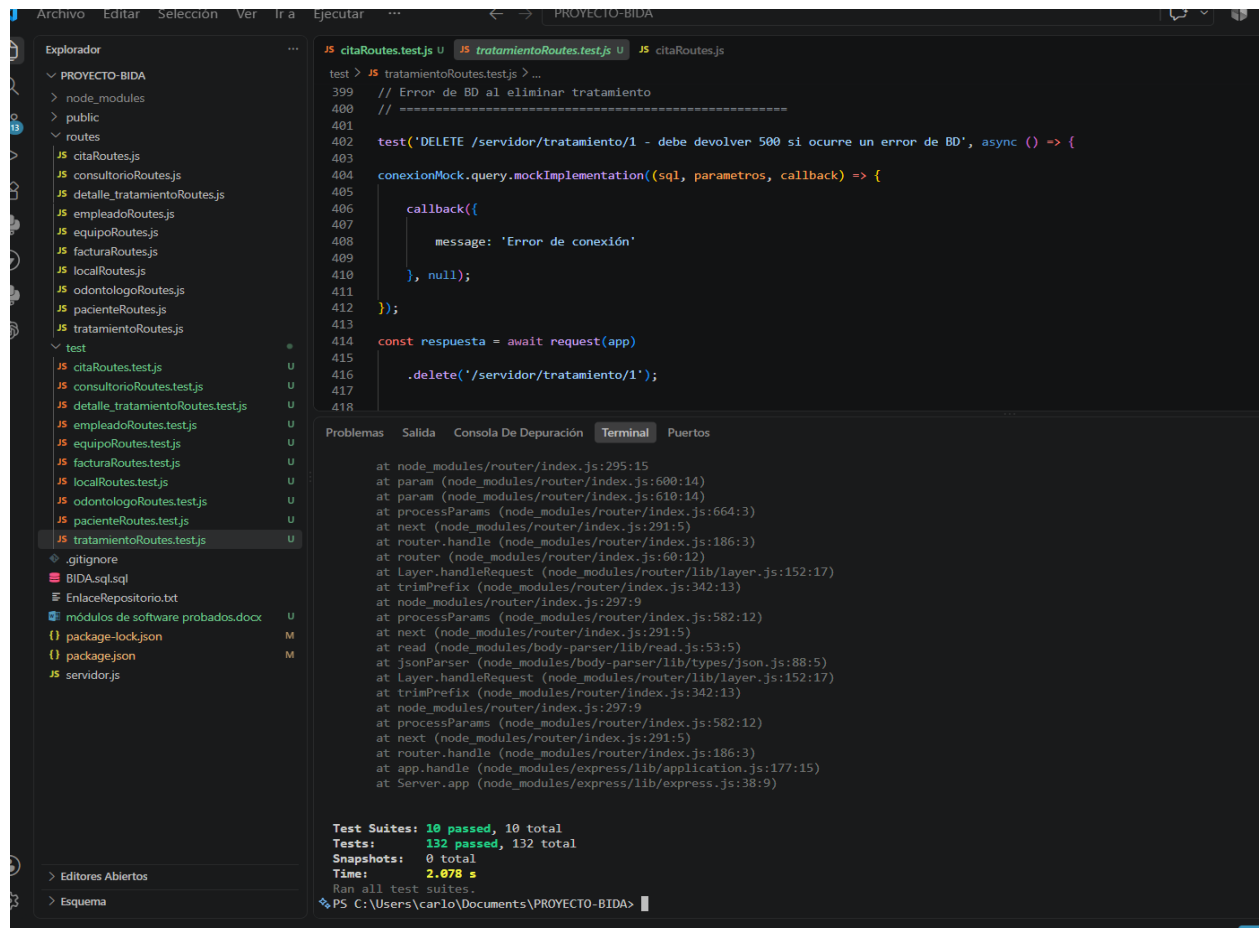
Test Suites: 10 passed, 10 total

Tests: 132 passed, 132 total

Snapshots: 0 total

Esto significa que las diez suites de pruebas fueron ejecutadas correctamente y que los 132 casos de prueba evaluados fueron aprobados.

El resultado general evidencia que las pruebas automatizadas implementadas para los módulos evaluados finalizaron satisfactoriamente.



18. EVIDENCIAS FINALES

Como evidencia del proceso de pruebas realizado se incluyen capturas de pantalla correspondientes a los archivos de pruebas y a la ejecución de los diferentes módulos mediante Jest.

Las evidencias permiten identificar los casos de prueba implementados para cada módulo y comprobar los resultados obtenidos durante su ejecución.

Se incluyen evidencias de los siguientes módulos:

Módulo de citas.

Módulo de tratamientos.

Módulo de facturación.

Módulo de pacientes.

Módulo de locales.

Módulo de odontólogos.

Módulo de empleados.

Módulo de detalle de tratamientos.

Módulo de consultorios.

Resultado general de las pruebas.

Las capturas se presentan como soporte del proceso de verificación realizado sobre el software desarrollado.